

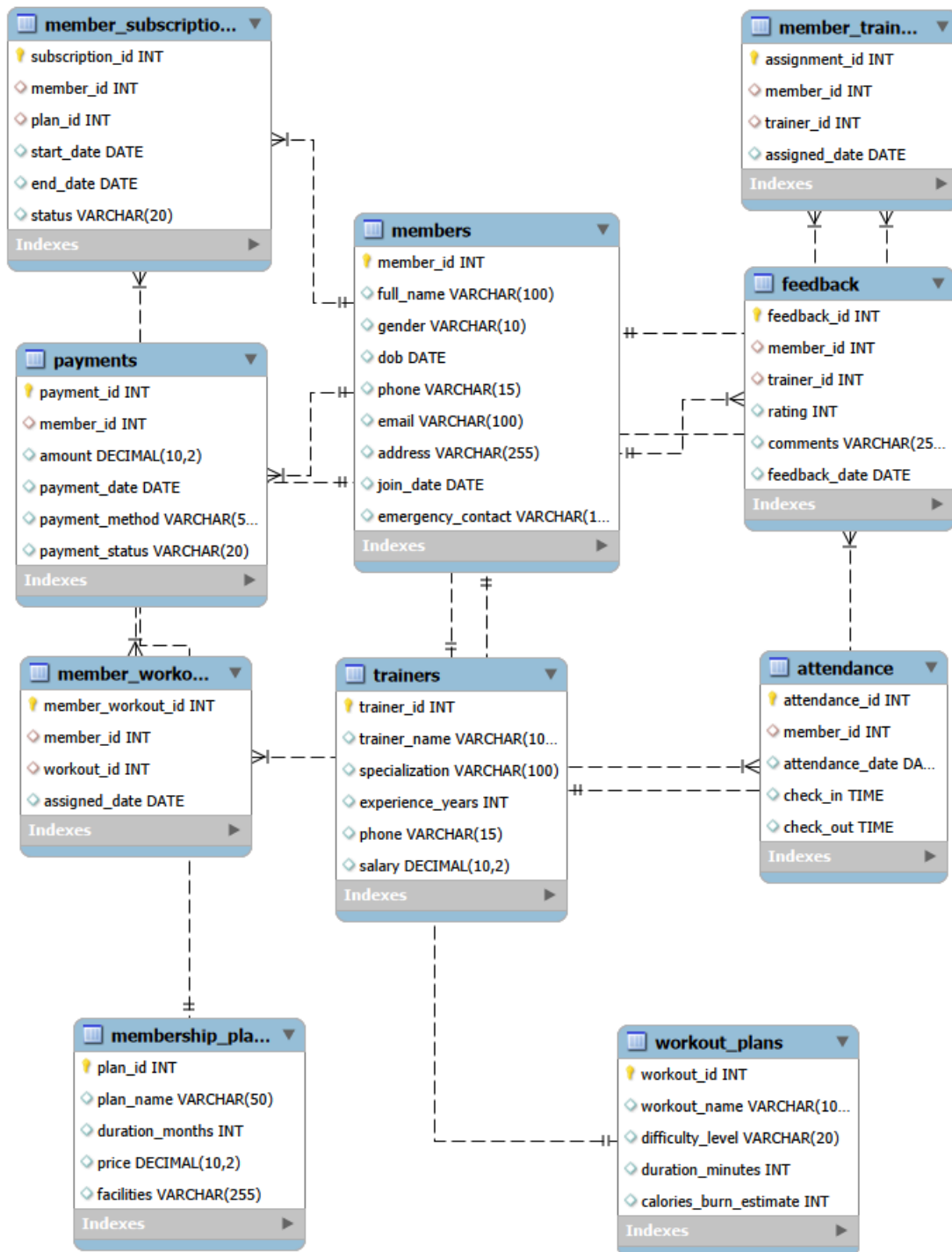
GYM MANAGEMENT SYSTEM

NAME : GIFTA JOICE P

GITHUB URL :

<https://github.com/gifta-testing/gym-system-management>

TABLE SCHEME



Basic Level

- Display all member details
- Display all trainer details
- Display all membership plans
- Display all payment details
- Display member names and phone numbers
- Display members from Chennai
- Display active subscriptions
- Find members whose names start with 'A'
- Find members whose names end with 'a'
- Display trainers ordered by salary descending
- Display first 5 members
- Display next 5 members
- Find total number of members
- Find highest trainer salary
- Find lowest trainer salary

Intermediate Level

- Find total revenue
- Find average payment amount
- Count members by subscription status
- Revenue by payment status
- Revenue greater than 5000
- Member, Plan and Payment Details
- LEFT JOIN
- RIGHT JOIN
- CROSS JOIN
- Highest paid trainer using subquery
- Members with paid payments
- Average trainer salary
- Create Member Report View
- Paid Payments
- Total Paid Amount

Advanced Level

- Trigger to prevent duplicate subscriptions
- Test Trigger
- Revenue Procedure
- Attendance Procedure
- Rank Trainers by Salary
- Row Number for Members
- Dense Rank for Payments
- TCL Commands
- DCL Commands

Duplicate Active Subscription Prevention Trigger

```
DROP TRIGGER IF EXISTS PreventDuplicateSubscription;
```

```
DELIMITER //
```

```
CREATE TRIGGER PreventDuplicateSubscription
BEFORE INSERT ON Member_Subscriptions
FOR EACH ROW
BEGIN
    DECLARE subCount INT;

    SELECT COUNT(*) INTO subCount
    FROM Member_Subscriptions
    WHERE member_id = NEW.member_id
    AND status = 'Active';

    IF subCount > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Member Already Has Active Subscription';
    END IF;
END //
```

```
DELIMITER ;
```

Test Query

```
INSERT INTO Member_Subscriptions
(member_id, plan_id, start_date, end_date, status)
VALUES
(2, 1, '2026-06-01', '2026-07-01', 'Active');
```

Payment Status Automation Trigger

DROP TRIGGER IF EXISTS PaymentAutomation;

DELIMITER //

CREATE TRIGGER PaymentAutomation
AFTER INSERT ON Payments
FOR EACH ROW
BEGIN

IF NEW.amount > 0 THEN

UPDATE Member_Subscriptions
SET status = 'Active'
WHERE member_id = NEW.member_id;

END IF;

END //

DELIMITER ;

Payment Status Automation

INSERT INTO Payments
(member_id, amount, payment_date, payment_method, payment_status)
VALUES
(1,1500,CURDATE(),'UPI','Paid');

Attendance Check Procedure

```
DROP PROCEDURE IF EXISTS CheckAttendance;
```

```
DELIMITER //
```

```
CREATE PROCEDURE CheckAttendance
```

```
(
```

```
    IN p_memberID INT
```

```
)
```

```
BEGIN
```

```
    DECLARE attendanceCount INT;
```

```
    SELECT COUNT(*)
```

```
    INTO attendanceCount
```

```
    FROM Attendance
```

```
    WHERE member_id = p_memberID;
```

```
    IF attendanceCount > 0 THEN
```

```
        SELECT 'Attendance Available' AS Status;
```

```
    ELSE
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'No Attendance Found';
```

```
    END IF;
```

```
END //
```

```
DELIMITER ;
```

Procedure Test

```
CALL CheckAttendance(1);
```

Revenue Procedure

DROP PROCEDURE IF EXISTS TotalRevenue;

DELIMITER //

CREATE PROCEDURE TotalRevenue()
BEGIN

 SELECT SUM(amount) AS Revenue
 FROM Payments
 WHERE payment_status = 'Paid';

END //

DELIMITER ;

Execute Procedure

CALL TotalRevenue();